🐳 Docker — Essential Commands

> Topic-only study notes • Taught in 3 stages + Interview Corner Demonstrated with the **hello-world** and **ubuntu** images.

Learner: Akshay **Date:** 6 August 2026

Essential Docker Commands

🟢 Beginner

Four buckets of commands: **images**, **running**, **listing**, **managing**.

```
# Images
docker pull ubuntu        # download an image
docker images             # list images
docker rmi hello-world    # remove an image

# Run containers
docker run hello-world    # prints a message and exits
docker run ubuntu         # starts, nothing to do, exits immediately

# List
docker ps                 # RUNNING containers
docker ps -a              # ALL containers (incl. stopped)

# Manage
docker stop <name|id>     # stop a running container
docker start <name|id>    # restart a stopped container
docker rm <name|id>       # delete a container
```

Why does `docker run ubuntu` exit instantly? A container lives only while its main process runs. `hello-world` prints and finishes → stops. Plain `ubuntu` has no long-running command → exits immediately. Use `-it ... bash` to keep it alive (next stage).

🟡 Working Developer

Go inside a container (most important):

```
docker run -it ubuntu bash    # -i interactive, -t tty, bash = shell inside
# prompt becomes root@<id>:/#  -> you're inside Ubuntu
```

```
ls
cat /etc/os-release           # confirms Ubuntu
apt update
exit                          # leaves container (it stops)
```

Daily-driver commands:

```
docker run -d --name myubuntu ubuntu sleep 1000  # detached, background
docker exec -it myubuntu bash    # jump INTO a running container
docker logs myubuntu             # view output/logs
docker stop myubuntu
docker start myubuntu
docker rm myubuntu               # delete container
docker rmi ubuntu                # delete image
```

`run` vs `exec`: `run` creates a *new* container from an image; `exec` runs a command inside an *already-running* container.

Cleanup:

```
docker rm $(docker ps -aq)    # remove ALL containers
docker system prune           # clean unused containers/networks/images
docker system prune -a        # also remove unused images
```

🔴 Expert

- **Lifecycle:** `created → running → paused/stopped → removed`. `docker run` = `docker create` + `docker start`. That's why a stopped container exists until `rm`.
- **`--rm`** auto-deletes on exit — best for throwaway tests: `docker run --rm -it ubuntu bash`.
- **Attach/detach:** `-d` runs detached; `docker attach <name>` re-connects. Inside an interactive container, `Ctrl+P` `Ctrl+Q` detaches **without stopping** it (vs `exit`, which stops it).
- **Inspect deeply:** `bash docker inspect myubuntu # full JSON: IP, mounts, env, config docker stats # live CPU/mem/network per container docker top myubuntu # processes inside the container docker diff myubuntu # files changed in the writable layer (copy-on-write!)`
- **Targeting:** commands accept name or a unique ID prefix (`docker stop a1b2`). `--name` saves typing.
- **Tags:** `docker run ubuntu` = `ubuntu:latest`; pin versions in real work (`ubuntu:22.04`).

🎯 Interview Corner

- **List running vs all?** `docker ps` (running) / `docker ps -a` (all).
- **`run` vs `exec`?** `run` = new container from image; `exec` = command inside a running container.
- **What is `-it`?** `-i` keeps STDIN open (interactive), `-t` allocates a pseudo-TTY. Together → interact with a shell inside.

- **Why does `docker run ubuntu` exit but `-it ubuntu bash` doesn't?** Container runs only while its main process is alive; `bash -it` keeps an interactive process running.
- **`stop` vs `rm` vs `rmi` ?** `stop` halts a running container; `rm` deletes a container; `rmi` deletes an image.
- **What does `--rm` do?** Auto-removes the container on exit (temporary/test containers).
- ⚠️ **Traps:** confusing `rm` (container) with `rmi` (image); forgetting `-a` when a container "vanishes" (it's stopped); saying `run` re-enters a container (that's `exec` / `start`).